E-Migration Assist — Architecture

E-Migration Assist — Architecture Document

Product owner: eRide Technologies **Document version:** 1.0 — May 5, 2026 **Status:** Pre-launch pilot

---

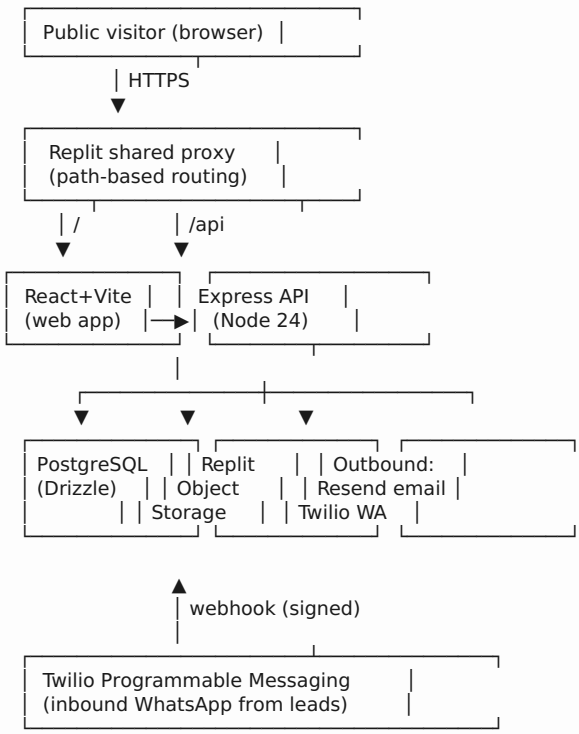
1. Purpose & scope

E-Migration Assist is the lead-capture and case-handover front-end of the eRide Technologies immigration platform. This document describes the system as currently deployed: code layout, runtime topology, data model, API surface, and the operational guarantees the system makes.

It does **not** cover the future Odoo / HubSpot / Jira / Paystack integrations shown on the high-level platform diagram — those are out-of-scope for the present codebase and will be layered on top of the APIs documented here.

---

2. System overview



Trust boundaries

Boundary	Surface	Auth
Public web	/, /assessment, /status, /thank-you/:ref	none (rate-limited)
Public API	POST /api/leads, GET /api/leads/:ref, GET /api/public/status/:ref, GET /api/stats/summary, POST /api/leads/:id/documents	none (rate-limited, PII-stripped responses)
Admin web	/admin, /admin/lead/:id, /admin/case/:caseId	x-admin-token (held in localStorage)
Admin API	GET /api/leads, GET /api/leads/:id (full), PATCH /api/admin/leads/:id, GET/PATCH /api/admin/cases/:caseId, GET /api/admin/leads/:id/messages, email update endpoints	x-admin-token header (constant-time compared to ADMIN_EMAIL_TOKEN)
Webhook	POST /api/webhooks/whatsapp	Twilio HMAC signature

3. Repository layout

This is a **pnpm monorepo**. Each artifact (deployable app) lives under artifacts/; shared libraries live under lib/.

```
artifacts-monorepo/
├── artifacts/
│   ├── api-server/          Express 5 backend (the only backend)
│   │   └── src/
│   │       ├── routes/      Route modules (one per concern)
│   │       ├── lib/         Cross-cutting helpers (auth, classification, cases, etc.)
│   │       └── index.ts     App composition + middleware wiring
│   ├── emigration-assist/   React + Vite + wouter web app
│   │   └── src/
│   │       ├── pages/       One file per route (home, assessment, admin, …)
│   │       ├── components/  Shared UI (BrandHeader, Disclaimer, DocumentUploader)
│   │       └── lib/         Frontend helpers (adminToken, leadStatus, caseStatus)
│   └── mockup-sandbox/      Design / variant preview (not production)
├── lib/
│   ├── db/                  Drizzle schemas (single source of truth)
│   ├── api-spec/            OpenAPI YAML + Orval codegen config
│   └── api-client-react/    Auto-generated React Query hooks
├── pnpm-workspace.yaml
└── replit.md                Living architecture summary
```

Contract-first rule: every endpoint that the frontend consumes synchronously is modelled in lib/api-spec/openapi.yaml and consumed via the generated React Query hooks. Endpoints that mutate admin-only data (PATCH lead, GET/PATCH case) are intentionally **not** modelled — the frontend uses raw fetch with the x-admin-token header. This is a deliberate convention to keep the OpenAPI spec free of admin-only endpoints that should never be discoverable.

4. Tech stack

Layer	Technology	Notes
Runtime	Node.js 24	Single version pinned across the workspace
Language	TypeScript 5.9	Strict mode in tsconfig.base.json
Backend framework	Express 5	With pino logger, CORS, body-parser, multer
Database	PostgreSQL	Replit-managed, accessed via DATABASE_URL
ORM	Drizzle ORM	Schemas in lib/db/src/schema/*.ts
Validation	Zod (+ drizzle-zod)	Same schemas drive client + server validation
Frontend framework	React 18 + Vite	@tailwindcss/vite plugin (Tailwind v4)
Routing (web)	wouter	Lightweight client-side router
Component library	shadcn/ui (Radix + Tailwind)	Components live in src/components/ui/
Data fetching	TanStack Query	Auto-generated hooks via Orval
API spec / codegen	OpenAPI 3 + Orval	Run pnpm --filter @workspace/api-spec run codegen
Object storage	Replit Object Storage	Bucket id in DEFAULT_OBJECT_STORAGE_BUCKET_ID
Email	Resend	API key via Replit integration
WhatsApp (outbound)	Twilio Programmable Messaging	TWILIO_* env vars
WhatsApp (inbound)	Twilio webhook	HMAC-validated
Bundling for prod	esbuild (CJS)	Server-side only

5. Data model

All tables live in lib/db/src/schema/. Every column follows a strict “text + Zod-validated enum” pattern instead of native PG enums — this lets us evolve enum values without writing migrations.

5.1 prelaunch_leads

The primary lead record. One row per assessment submission.

Column	Type	Notes
id	UUID PK	
reference_number	TEXT UNIQUE	Public-safe identifier EMA-XXXXXXXX-XXXX
full_name, email	TEXT	PII — never returned on public endpoints
nationality, country_of_residence	TEXT	
whatsapp_number_raw, whatsapp_number_canonical	TEXT	Canonical form is the dedup key
whatsapp_number_country	TEXT	ISO 2-letter
preferred_contact_channel	TEXT	email | whatsapp
immigration_situation, urgency, details	TEXT	Assessment answers
consent_accepted	BOOLEAN	+ consent_timestamp
lead_status	TEXT	Enum below
lead_priority	TEXT	high | medium | low
lead_score	INT	Computed
lead_category	TEXT	Visa expiry / overstay / lost docs / other
internal_classification	TEXT	Operator-only
admin_notes	TEXT	Operator-only
created_at, updated_at	TIMESTAMP	

5.2 lead_documents

Optional uploads attached to a lead.

Column	Type	Notes
id	UUID PK	
lead_id	UUID FK	
filename, mime_type, size_bytes		Validated server-side (magic bytes + size)
storage_key	TEXT	Object Storage key (UUID-prefixed)
created_at	TIMESTAMP	

5.3 lead_engagements

Audit log of every outbound communication attempt.

Column	Type	Notes
id	UUID PK	
lead_id	UUID FK	
channel	TEXT	email | whatsapp
kind	TEXT	confirmation | manual_update | etc.
status	TEXT	pending | sent | failed
provider_message_id	TEXT	Resend / Twilio id
error	TEXT	Provider error if failed
created_at	TIMESTAMP	Cooldown window enforced per channel

5.4 lead_cases

Created when a lead reaches converted. Idempotent on lead_id.

Column	Type	Notes
id	UUID PK	
lead_id	UUID **UNIQUE** FK	Idempotency primitive
reference_number	TEXT	Snapshot of lead's ref at conversion
status	TEXT	Case lifecycle — see §6.2
created_at, updated_at	TIMESTAMP	

5.5 case_messages

Inbound WhatsApp messages associated with a lead.

Column	Type	Notes
id	UUID PK	
lead_id	UUID FK	
direction	TEXT	inbound
channel	TEXT	whatsapp
body	TEXT	Raw message body
intent	TEXT	task_complete_signal for "done/uploaded/sent" keywords
provider_message_id	TEXT	Twilio SID
received_at	TIMESTAMP	

6. Lifecycle state machines

Both state machines below are **forward-only** and enforced atomically in the SQL UPDATE ... WHERE predicate to close the TOCTOU race where two concurrent operators could regress one another's writes.

6.1 Lead funnel (prelaunch_leads.lead_status)

Source of truth: artifacts/api-server/src/lib/classification.ts LEAD_STATUS_VALUES.

new ⟶ reviewing ⟶ contacted ⟶ qualified ⟶ ready_for_case ⟶ converted ⟶ closed

- Forward jumps are allowed (e.g. new → qualified).
- Same-status PATCH is a 200 no-op (safe for optimistic-UI retries).
- Backward PATCH → **HTTP 409** with the canonical order in the message.
- Legacy/unknown statuses are passed through (never lock a row).
- Frontend mirror lives in artifacts/emigration-assist/src/lib/leadStatus.ts and disables backward dropdown options with a tooltip.

6.2 Case lifecycle (lead_cases.status)

Source of truth: artifacts/api-server/src/lib/caseStatus.ts CASE_STATUS_VALUES.

initiated ⟶ in_review ⟶ documents_requested ⟶ submitted ⟶ closed

Same atomic-WHERE forward-only guard as the lead funnel; same 409 response shape; same frontend mirror in artifacts/emigration-assist/src/lib/caseStatus.ts.

6.3 Lead → case conversion

When PATCH /api/admin/leads/:id results in lead_status = "converted":

1. ensureCaseForLead(leadId) runs an INSERT ... ON CONFLICT (lead_id) DO NOTHING RETURNING ... against lead_cases.
2. If 0 rows returned (case already exists), a fallback SELECT fetches the existing one.

3. The `caseId` is included in the PATCH response (and in every Lead serialization via LEFT JOIN).
4. The handler runs **on every PATCH that resolves to `converted`** — not only on status change — so notes-only edits on already-converted leads still surface `caseId`.
5. PATCH fails with HTTP 500 if case creation errors (no silent fallback).

The unique index on `lead_cases.lead_id` is the durable idempotency primitive — concurrent PATCHes can race freely; only one row will ever exist per lead.

---

7. API surface (current)

Public (no auth)

Method	Path	Purpose
GET	/api/healthz	Liveness probe
GET	/api/stats/summary	Counts for the landing page (no PII)
POST	/api/leads	Submit assessment → creates lead
GET	/api/leads/:reference	Public-safe lead view (no email/whatsapp/notes/status)
GET	/api/public/status/:reference	{ referenceNumber, publicLabel, createdAt, documentsUploaded } only
POST	/api/leads/:id/documents	Upload a document (multipart, validated)

Admin (`x-admin-token` required)

Method	Path	Purpose
GET	/api/leads	List leads (full PII) with filters
GET	/api/leads/:id	Single lead with PII + caseId
PATCH	/api/admin/leads/:id	Update status / priority / notes; auto-creates case if status reaches `converted`
GET	/api/admin/cases/:caseId	Case + embedded lead snapshot
PATCH	/api/admin/cases/:caseId	Advance case lifecycle (forward-only)
GET	/api/admin/leads/:id/messages	Inbound WhatsApp messages for a lead
POST	/api/admin/leads/:id/email	Send manual update email

Webhook

Method	Path	Purpose
POST	/api/webhooks/whatsapp	Twilio inbound message ingress (signed)

---

8. Cross-cutting concerns

8.1 Authentication

- **Admin token (ADMIN_EMAIL_TOKEN)** is the only auth primitive. It's compared in **constant time** in requireAdminToken() (api-server/src/lib/adminAuth.ts).
- The middleware fails **closed (503)** if ADMIN_EMAIL_TOKEN is unset — the system cannot accidentally run without admin auth.
- The token is held in browser localStorage under admin_token; all admin API calls send it as x-admin-token. On 401, the token is cleared and the user is bounced to a sign-in card.

8.2 Rate limiting

Express middleware applies separate rate limits to public and admin endpoints. The public lookup endpoints (/api/leads/:ref, /api/public/status/:ref) have particularly tight limits to defeat reference-number enumeration.

8.3 PII handling

- Public endpoints use a **separate serializer** that strips email, WhatsApp, internal classification, admin notes, lead status, lead priority, lead score, and caseId.
- Logs use req.log (pino) with PII redaction.
- console.log is forbidden in server code.

8.4 Document storage

- Files uploaded via multipart go through magic-byte validation (file-type) and a 10 MB size cap before reaching Object Storage.
- Storage keys are UUID-prefixed so there is no enumeration vector.
- Downloads are proxied by the API server (never direct from Object Storage) so admin auth is enforced and access logged.

8.5 Engagement (outbound comms)

- Channel-agnostic sendMessage() gateway; backends are Resend (email) and Twilio (WhatsApp).
- Sending is **non-blocking** on lead submission (the POST response doesn't wait for SMTP/WA delivery).
- Per-channel **1-minute cooldown** prevents accidental flooding on resubmission.
- Emails are screened for forbidden phrases (legal-jargon guard) before send.

8.6 Inbound WhatsApp

- Webhook returns 200 immediately, then queues the message store.
- Twilio HMAC signature is validated against WHATSAPP_APP_SECRET (when present) — set this secret to enable signature enforcement in production.
- Deterministic keyword detection (done, uploaded, sent) tags messages with intent='task_complete_signal' so operators can sort their inbox.

8.7 Branding

- Mono-mode dark navy + teal palette in artifacts/emigration-assist/src/index.css (:root + .dark alias).
- Shared BrandHeader component on every page, sourcing the logo from public/eride-logo-light.png (derived from the eRide PDF asset).

---

9. Deployment & ops

- **Hosted on:** Replit Deployments (*.replit.app, custom domain optional).
- **Build:** pnpm run build (esbuild bundles the API to CJS; Vite builds the static web).
- **Runtime topology:** API and web run as separate workflows behind the Replit shared path-based proxy. The web app uses import.meta.env.BASE_URL for API calls (do not hard-code root-relative paths).

- **Secrets** (managed via Replit env vars):
  - DATABASE_URL, SESSION_SECRET — required
  - ADMIN_EMAIL_TOKEN — required (server fails closed without it)
  - RESEND_API_KEY — required for email
  - TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM — required for outbound WA
  - WHATSAPP_APP_SECRET, WHATSAPP_VERIFY_TOKEN — recommended for inbound webhook hardening
  - DEFAULT_OBJECT_STORAGE_BUCKET_ID, PUBLIC_OBJECT_SEARCH_PATHS, PRIVATE_OBJECT_DIR — Object Storage
- **Database migrations:** Drizzle schema is the source of truth; `pnpm db:push` against the dev DB. Production schema sync uses the same flow (see the `database` skill).

---

10. Architectural decisions (and their reasons)

#	Decision	Rationale
AD-1	Forward-only state machines enforced in SQL `WHERE`	Closes the TOCTOU race where two operators read-then-write concurrently
AD-2	Lead status & case status as `text` columns, not PG enums	Adding a new stage is an array append + frontend mirror update — no migration
AD-3	`lead_cases.lead_id` UNIQUE is the only idempotency primitive for conversion	DB enforces "one case per lead" regardless of how many concurrent PATCHes race
AD-4	Admin-only mutations are NOT in OpenAPI; use raw fetch from frontend	Keeps the discoverable spec free of operator-only endpoints
AD-5	Public serializers never include `caseId` or status fields	Defense-in-depth: even if admin auth is misconfigured, public routes can't leak the funnel
AD-6	`requireAdminToken` fails closed (503) if env var missing	Cannot accidentally deploy with admin auth disabled
AD-7	Outbound comms are non-blocking on lead submission	Lead capture must always succeed even if Resend/Twilio are degraded
AD-8	Mono-mode dark navy theme (no light/dark toggle)	Brand consistency; matches eRide product mockups

---

11. Known limitations / future work

- **Conversion staging** is UI-only: the server allows `converted` from any prior status. If business rules require `ready_for_case → converted` only, add a single allow-list check in `PATCH /api/admin/leads/:id`.
- **No background job runner** — outbound delivery happens inline (with cooldown). At scale, move to a queue (BullMQ / Cloud Tasks).
- **No multi-tenant scoping** — all admin tokens see all leads. Splitting into firms/teams will require a `tenants` table and per-row authorization.
- **No audit log of admin actions** — `lead_engagements` covers comms but not field edits. A `lead_audit` table is planned.
- **No structured case events** — `case_messages` only stores inbound WA. A `case_events` log unifying status changes, doc requests, and

operator actions would feed an activity timeline.
- **Future integrations** (per platform diagram): Odoo, HubSpot, Jira, Paystack, AWS S3 mirroring. None are implemented today.